

CKA: Helm Basics (2025 Updates)

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Installation and configuration

This lesson explains the steps required to install Helm, a package manager for Kubernetes, on Linux systems.

This lesson explains the steps required to install Helm, a package manager for Kubernetes. Before starting, ensure you have a functioning Kubernetes cluster and that `kubectl` is properly configured. Verify that your `kubeconfig` file contains the correct credentials to access your Kubernetes cluster.

Make sure your system has access to a working Kubernetes cluster and that `kubectl` is set up correctly. A valid `kubeconfig` file is essential for Helm to connect to your cluster.

Helm is compatible with Linux, Windows, and macOS. This guide focuses on installing Helm on Linux systems.

Installing Helm on Linux

Using Snap

If your system supports Snap, you can install Helm using the command below. The `--classic` option offers a more relaxed sandbox environment, enabling Helm to easily locate your `kubeconfig` file in your home directory and connect to your Kubernetes cluster:

```
```bash theme={null} sudo snap install helm --classic
```

### ### Using APT (Debian/Ubuntu)

For apt-based systems such as Debian or Ubuntu, follow these steps to add the Helm package repository and

```
```bash theme={null}
curl https://baltocdn.com/helm/signing.asc | sudo apt-key add -
sudo apt-get install apt-transport-https --yes
echo "deb https://baltocdn.com/helm/stable/debian/ all main" | sudo tee /etc/apt/sources.list.d/helm-st
sudo apt-get update
sudo apt-get install helm
```

Using PKG

On systems that support PKG, you can install Helm with the following command:

```
bash theme={null} pkg install helm
```

For the most current installation procedures, always refer to the official Helm documentation.

Conclusion

With Helm installed, you are now ready to explore its capabilities in your lab environment. This guide provided step-by-step instructions for various Linux distributions to help streamline your Helm installation process.

Happy Helming!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

A quick note about Helm2 vs Helm3

This article explains the differences between Helm 2 and Helm 3, focusing on improvements in usability and security for Kubernetes charts.

Helm has evolved significantly since its inception, and understanding the differences between Helm 2 and Helm 3 is crucial when working with Kubernetes charts. This article provides a brief history of Helm and highlights the key improvements introduced in Helm 3, offering insights into its enhanced usability and security.

A Brief History of Helm

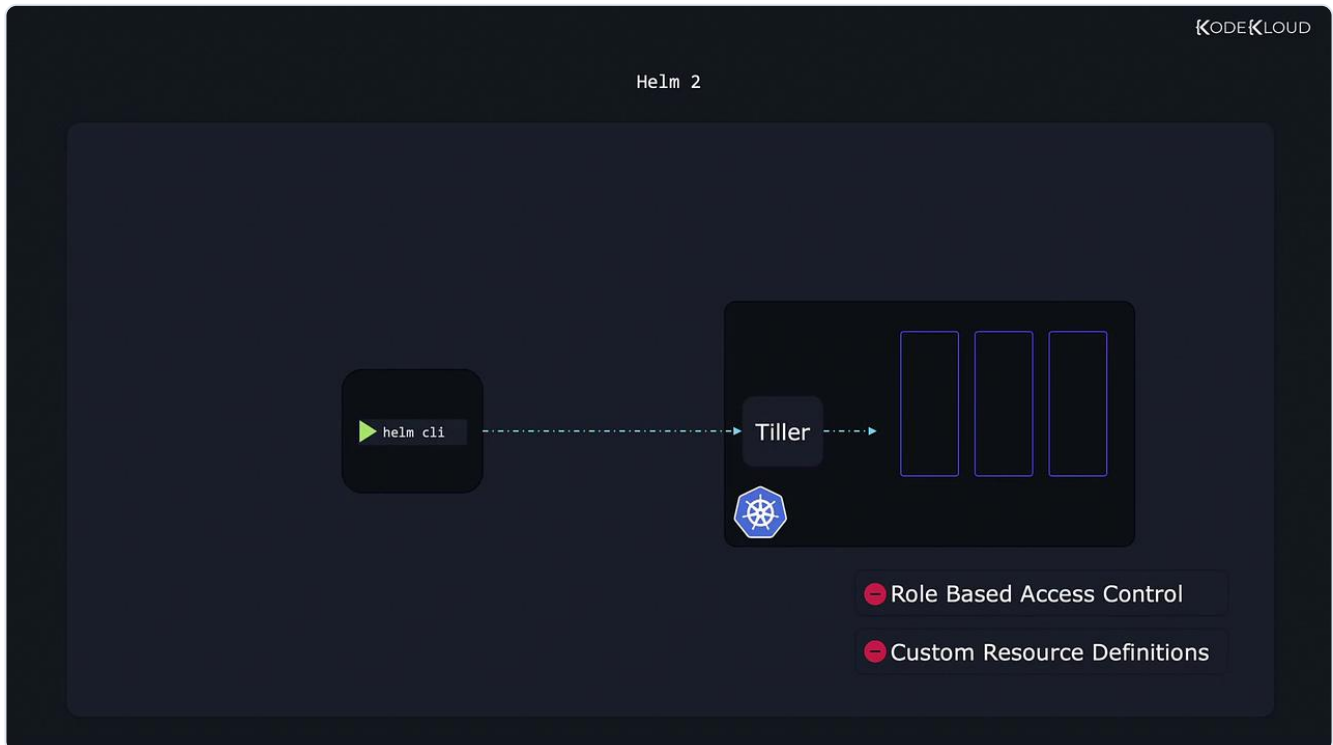
Helm's journey began in February 2016 with Helm 1.0, followed by Helm 2.0 in November 2016. The major milestone was achieved with the release of Helm 3.0 in November 2019. As Kubernetes itself advanced, Helm matured to better leverage Kubernetes functionalities, resulting in a more robust and secure package management experience.



Helm 2: The Role of Tiller

In the era of Helm 2, Kubernetes limitations such as the absence of role-based access control (RBAC) and custom resource definitions (CRDs) meant that an additional component, Tiller, was required. Tiller acted as an intermediary between the Helm CLI and the Kubernetes cluster, managing the installation and upgrade of charts. However, this approach introduced several challenges:

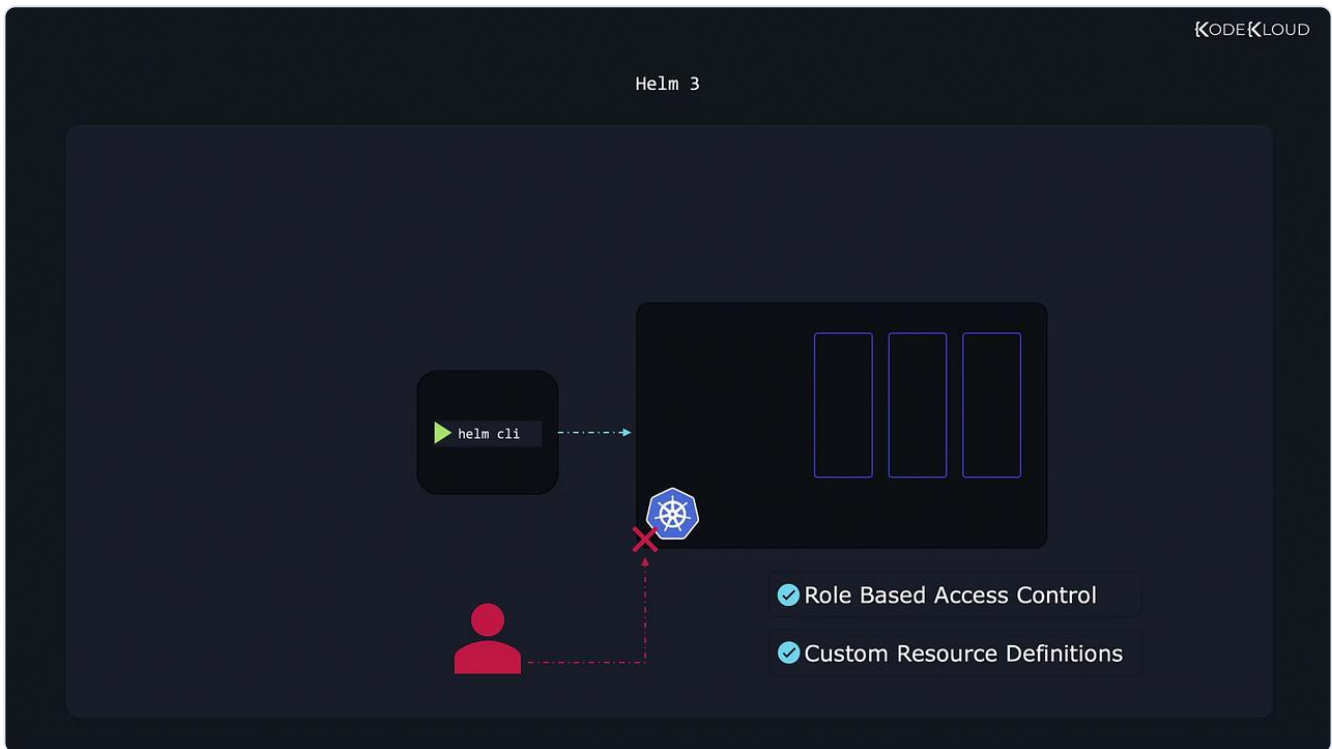
- **Complexity:** The interaction between the CLI and Tiller added an extra layer to the deployment process.
- **Security Risks:** Tiller operated with broad privileges, often referred to as running in "God mode," increasing the risk of potential security issues.



Using Tiller in Helm 2 means that any misconfiguration could lead to elevated security risks due to its extensive privileges.

Helm 3: Simplified and Secure

Helm 3 brings notable improvements by eliminating the need for Tiller. Instead, the Helm client now communicates directly with Kubernetes, leveraging its native RBAC capabilities. This change simplifies the architecture and enhances security, as every Helm action is subject to the same RBAC permissions applied when using `kubectl`.



Key Differences Between Helm 2 and Helm 3

Feature	Helm 2	Helm 3
Intermediary Component	Requires Tiller	No intermediary; direct communication
Security Model	Elevated risk due to broad privileges	Enhanced security leveraging Kubernetes RBAC
Rollback Mechanism	Basic revision comparison	Three-way strategic merge patch

Improved Rollback and Upgrade Process in Helm 3

One of the significant advancements in Helm 3 is the handling of rollbacks through a three-way strategic merge patch, resembling a snapshot mechanism. Consider the example of deploying a WordPress website using a Helm chart:

1. **Installation:** When you install the chart, Helm creates revision 1.
2. **Upgrade:** After changing configurations, such as upgrading the WordPress image, Helm creates revision 2.
3. **Rollback:** If needed, reverting to revision 1 will restore the previous state.

Below is an example of the process:

```
```bash
theme={null} $ helm install wordpress $ helm upgrade wordpress
```

```
Revision Changes in YAML

```yaml theme={null}
# Revision 1: Using an older WordPress image
containers:
  - image: wordpress:4.8-apache
```

```
```yaml theme={null}
```

## Revision 2: After upgrading to a newer WordPress image

---

```
containers: - image: wordpress:5.8-apache
```

To roll back to revision 1:

```
```bash theme={null}
$ helm rollback wordpress
```

With each significant change invoked via Helm commands, a new revision is created. For instance, installation creates revision 1, an upgrade creates revision 2, and a rollback may create revision 3 representing the restored state.

Helm 3's intelligent three-way comparison considers the following:

- The previous chart revision,
- The desired chart state,
- The live state of Kubernetes objects. This approach ensures that discrepancies—such as manual changes using `kubectl`—are correctly reconciled.

Handling Manual Changes and Upgrades

In Helm 2, if a user manually modified Kubernetes objects (e.g., using `kubectl set image`) after deployment, these changes were not recorded in Helm's revision history, meaning Helm might not detect any differences during a rollback. In contrast, Helm 3 compares the live state against both the current and desired revisions. This ensures:

- Manual modifications outside of Helm are preserved during upgrades.
- Overwritten configurations are avoided unless explicitly intended.

Conclusion

The transition from Helm 2 to Helm 3 marks a significant improvement in Kubernetes deployment management. By removing Tiller and implementing a more robust rollback mechanism via a three-way strategic merge patch, Helm 3 ensures enhanced security, simplified architecture, and greater reliability during upgrades.

For more detailed information on Helm and Kubernetes, check out the following resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [Helm Documentation](#)

This guide should help you understand the evolution of Helm and leverage Helm 3's features for a more secure and streamlined deployment experience.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Helm Components

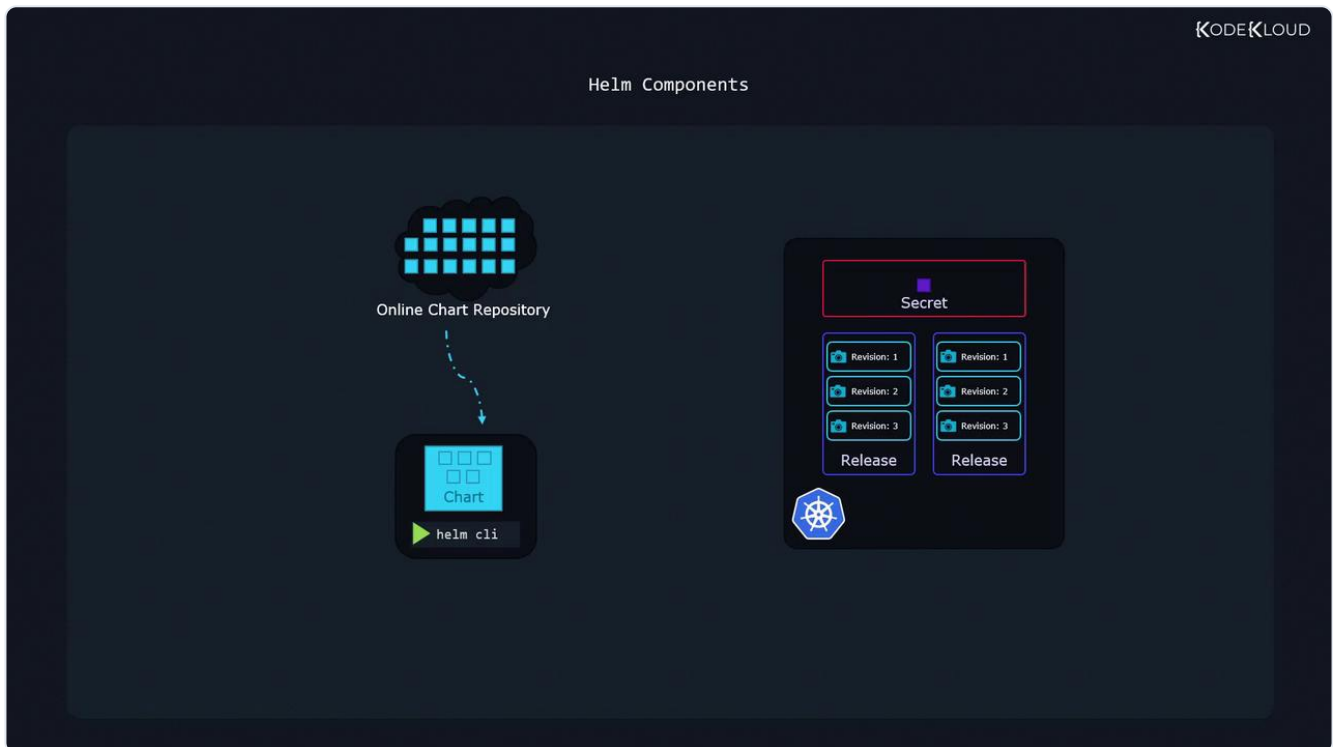
This article provides an in-depth look at Helm components, including its structure, concepts, and key elements for managing Kubernetes applications effectively.

In this article, we dive into the components of Helm, providing an in-depth look at its structure, concepts, and key elements essential for managing Kubernetes applications effectively.

Helm is primarily composed of a command-line tool installed locally, which you can use to install, upgrade, or roll back releases. Charts—collections of files containing instructions for creating Kubernetes objects—are used by Helm to deploy applications. When you deploy a chart to your Kubernetes cluster, Helm creates a release, representing a specific installation of the application. Each release may have multiple revisions, capturing changes like image upgrades, replica adjustments, or configuration updates.

Similar to how [Docker Hub](#) hosts container images or [Vagrant Cloud](#) provides boxes, public repositories host Helm charts. These repositories allow you to quickly download and deploy applications on your cluster.

Helm stores metadata—including information about installed releases, used charts, and revision history—directly into your Kubernetes cluster as secrets. This ensures that metadata is persistent and accessible to all team members, facilitating seamless upgrades and maintenance operations.



Helm maintains a comprehensive record of every action performed within the cluster, enabling precise tracking and management.

Charts and Templating

Charts are collections of files that contain the instructions for creating Kubernetes objects, making them the backbone of Helm deployments. This article uses two application examples to illustrate various concepts:

1. **HelloWorld Application:** A simple Nginx-based web server with a service for exposure.
2. **WordPress Site:** A more complex application deployment.

The HelloWorld example demonstrates fundamental concepts effectively. In this example, two Kubernetes objects are deployed: a Deployment and a Service. Although similar to standard Kubernetes definitions, you'll notice that parameters like the image name and replica count are templated. These templated values are defined in a separate `values.yaml` file, which allows you to customize your deployment with minimal changes.

Below is an example of a simple HelloWorld chart:

```
```yaml theme={null}
```

## service.yaml

```
apiVersion: v1 kind: Service metadata: name: hello-world spec: type: NodePort ports: - port: 80 targetPort: http protocol: TCP name: http selector: app: hello-world
```

```
```yaml theme={null}
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: nginx
          image: {{ .Values.image.repository }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
```

```
```yaml theme={null}
```

## values.yaml

---

replicaCount: 1 image: repository: nginx

Rather than building charts from scratch, you can download pre-built charts from public repositories. Customizing Helm charts

For more complex applications like WordPress, charts can include multiple files and advanced templating for

A more advanced templating snippet within a Deployment might look like this:

```
```yaml theme={null}
apiVersion: {{ include "common.capabilities.deployment.apiVersion" . }}
kind: Deployment
metadata:
  name: {{ include "common.names.fullname" . }}
  namespace: {{ .Release.Namespace | quote }}
  labels:
    {{- include "common.labels.standard" . | nindent 4 }}
    {{- if .Values.commonLabels }}
    {{- include "common.tplvalues.render" (dict "value" .Values.commonLabels "context" $) | nindent 4 }}
    {{- end }}
    {{- if .Values.commonAnnotations }}
  annotations:
    {{- include "common.tplvalues.render" (dict "value" .Values.commonAnnotations "context" $) | nindent 4 }}
    {{- end }}
spec:
  selector:
    matchLabels: {{- include "common.labels.matchLabels" . | nindent 6 }}
    {{- if .Values.updateStrategy }}
  strategy: {{- toYaml .Values.updateStrategy | nindent 4 }}
    {{- end }}
    {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
    {{- end }}
```

Releases and Multiple Installations

When applying a chart to your cluster, Helm creates a release—a distinct instance of the application. This approach allows you to deploy multiple separate instances of the same chart with unique release names. For example, you can deploy two different WordPress sites using separate releases:

```
```bash theme={null}
```

## helm install [release-name] [chart]

```
$ helm install my-site bitnami/wordpress
```

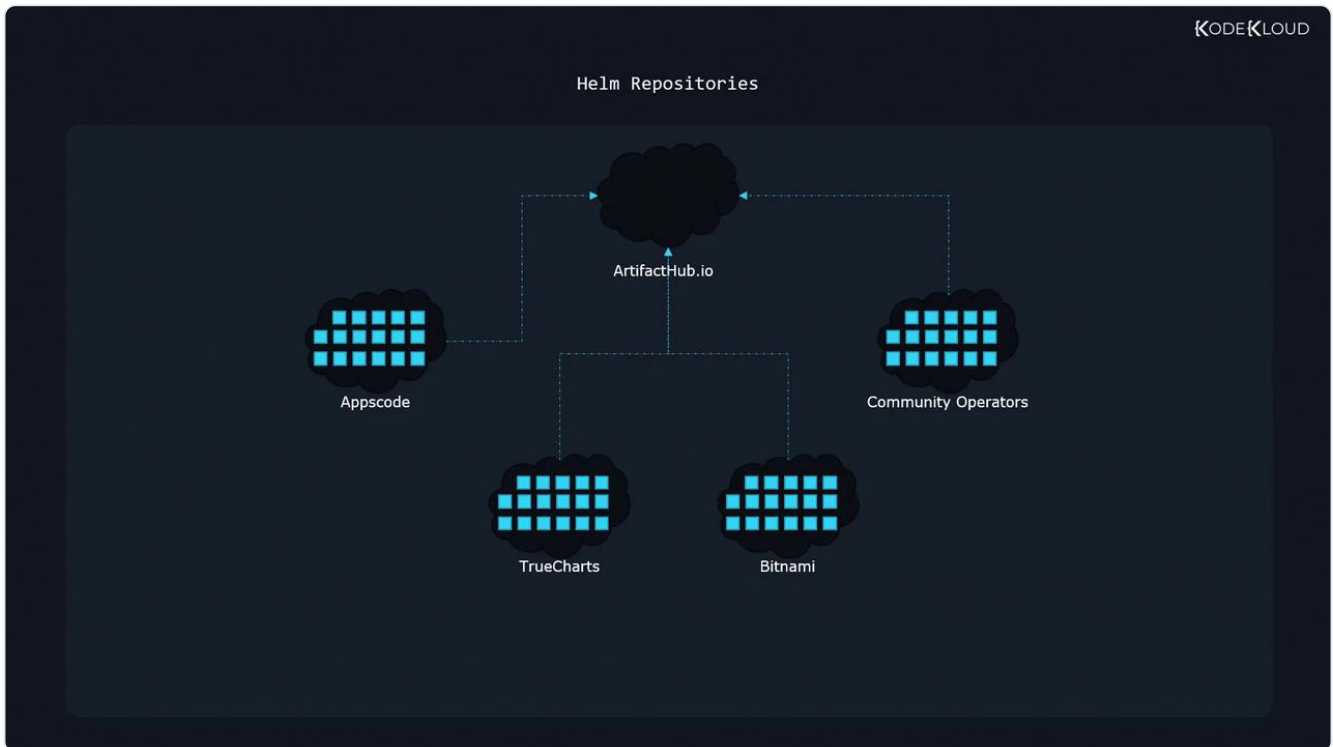
```
$ helm install my-SECOND-site bitnami/wordpress ```
```

Each release is tracked independently, even if they are based on the same chart. This functionality is particularly useful when maintaining different environments, such as having one release for a public-facing website and another for development purposes. Experimentation in the development release can then inform upgrades or changes to the production version.

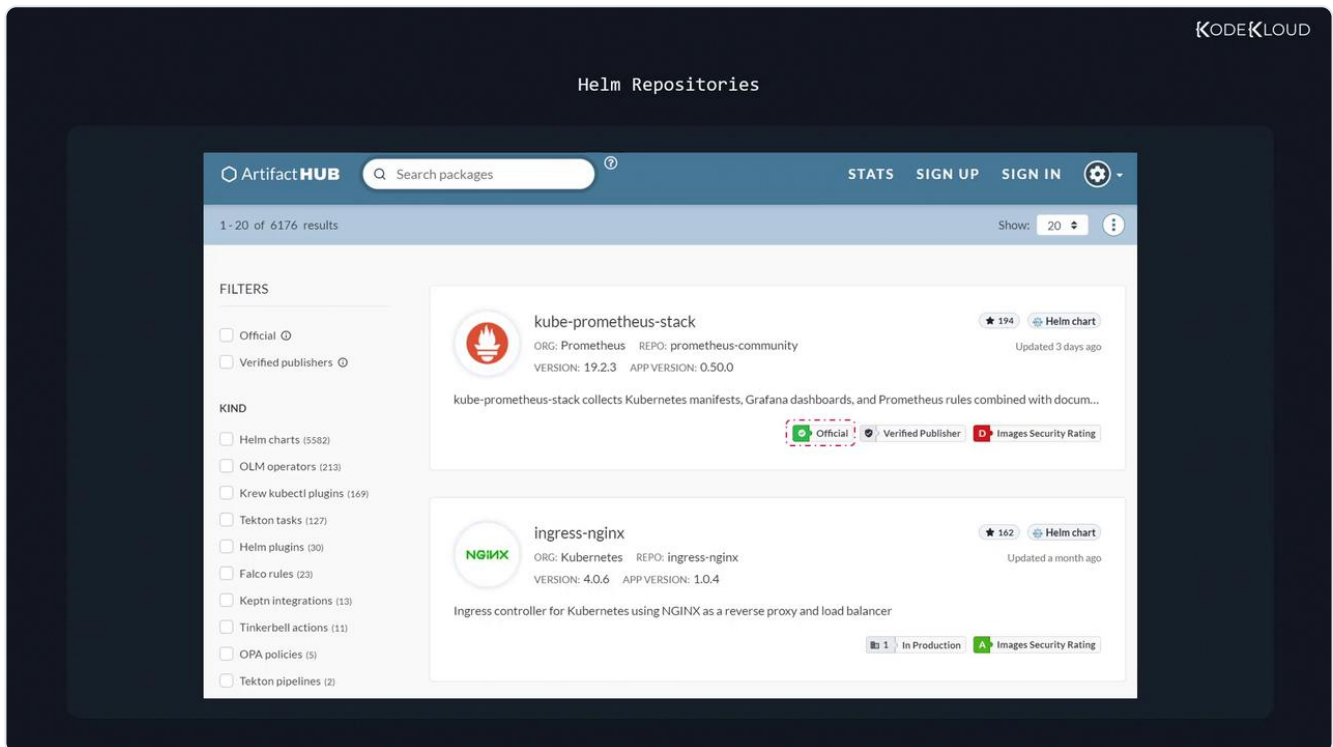
## Helm Repositories and Artifact Hub

Beyond our basic examples, Helm charts are available for a wide range of applications—from Redis to Prometheus—across numerous public repositories. Providers such as Appscore, Community Operators, TrueCharts, and Bitnami host charts in their repositories, making it easy to deploy various applications.

Instead of visiting multiple repositories separately, you can use the centralized [Artifact Hub](#) to search for and manage charts. Artifact Hub currently features over 6,300 packages and highlights charts published by official developers with verified publisher badges for added trustworthiness.



Artifact Hub also provides a searchable interface to help you quickly find the charts you need:



In upcoming lessons, we will explore chart installation and customization in greater detail. Continue following our guide to deepen your knowledge of the practical applications of Helm in Kubernetes.

Happy Helm-ing, and see you in the next lesson!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Helm charts

This article explores Helm Charts for managing Kubernetes applications, detailing their structure, usage, and deployment processes.

In this lesson, we explore Helm Charts—a powerful tool for managing Kubernetes applications. Helm simplifies tasks such as installing, upgrading, rolling back, and uninstalling applications by automating the complex steps required to achieve the desired state.

Helm Charts act as comprehensive instruction manuals for your deployments. Each chart is a structured collection of files that define an application's configuration and behavior on Kubernetes. For example, the parameters in the values.yaml file enable operators to customize configurations without modifying the underlying templates.

Use Helm's templating syntax (e.g., `{{ .Values.replicaCount }}`) in your manifests to keep configuration flexible and reusable. All dynamic values are defined in the values.yaml file.

Below is a simple example of Helm template files that create two Kubernetes objects—a Deployment and a Service. The Deployment manages a set of Pods based on a specified image, and the Service exposes these Pods as a NodePort service:

```
``yaml theme={null} apiVersion: v1 kind: Service metadata: name: hello-world spec: type: NodePort ports: - port: 80 targetPort: http protocol: TCP name: http selector: app: hello-world
```

```
apiVersion: apps/v1 kind: Deployment metadata: name: hello-world spec: replicas: {{ .Values.replicaCount }} selector: matchLabels: app: hello-world template: metadata: labels: app: hello-world spec: containers: - name: hello-world image: "{{ .Values.image.repository }}" ports: - name: http containerPort: 80 protocol: TCP
```

To install this chart, run the following command:

```
``bash theme={null}
$ helm install hello-world
```

Notice that values like the image repository and replica count are not hardcoded. Instead, they utilize Helm's templating syntax, which references configurations defined in the values.yaml file. This approach allows you to easily adjust parameters without directly editing the template files.

## Chart Metadata

Every Helm chart includes a Chart.yaml file that contains essential metadata, such as:

- **API Version:** For Helm 3, set to `v2` (Helm 2 charts use `v1` or omit this field).
- **App Version:** Indicates the version of the application deployed.
- **Chart Version:** Tracks the version of the chart itself, independent of the application version.
- **Name and Description:** Provide identification and a brief summary of the chart.
- **Type:** Specifies whether the chart is for an application (default) or is a library chart.
- **Dependencies:** Declare any external charts that the chart relies on.
- **Additional Fields:** Optional fields like keywords, maintainers, home, and icon help with discovery and branding.

Below is an example that combines Kubernetes manifest templates with chart metadata:

```
``yaml theme={null}
```

## Service and Deployment templates

```
apiVersion: v1 kind: Service metadata: name: hello-world spec: type: NodePort ports: - port: 80 targetPort: http
protocol: TCP name: http selector: app: hello-world
```

```
apiVersion: apps/v1 kind: Deployment metadata: name: hello-world spec: replicas: {{ .Values.replicaCount }}
selector: matchLabels: app: hello-world template: metadata: labels: app: hello-world spec: containers: - name:
hello-world image: "{{ .Values.image.repository }}" ports: - name: http containerPort: 80 protocol: TCP
```

## values.yaml snippet

```
replicaCount: 1 image: repository: nginx
```

## Chart.yaml snippet

```
apiVersion: v2 appVersion: "1.16.0" name: hello-world description: A web application type: application
```

Again, the chart can be installed with:

```
``bash theme={null}
$ helm install hello-world
```

## Example: WordPress Chart

For a more complex use case, consider a WordPress chart that depends on additional services like MariaDB. Below is an example of a Chart.yaml file for a WordPress deployment:

```
``yaml theme={null} apiVersion: v2 appVersion: 5.8.1 version: 12.1.27 name: wordpress description: Web publishing platform for building blogs and websites. type: application dependencies: - condition: mariadb.enabled name: mariadb repository: https://charts.bitnami.com/bitnami version: 9.x.x keywords: - application - blog - wordpress maintainers: - email: containers@bitnami.com name: Bitnami home: https://github.com/bitnami/charts/tree/master/bitnami/wordpress icon: https://bitnami.com/assets/stacks/wordpress/img/wordpress-stack-220x234.png
```

### ## Helm Chart Directory Structure

A typical Helm chart directory includes the following components:

- \* **templates/**: Contains all the manifest templates (e.g., Deployment, Service).
- \* **values.yaml**: Defines default configuration values.
- \* **Chart.yaml**: Holds metadata about the chart.
- \* **charts/**: Optionally includes dependent charts (e.g., the MariaDB chart for WordPress).
- \* Other optional files such as **LICENSE** or **README** for additional information.

### ## Deploying a Chart from a Repository

To deploy the WordPress chart from the Bitnami repository, execute the following commands:

```
``bash theme={null}
$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm install my-release bitnami/wordpress
```

You can verify your installation using similar commands:

```
plaintext theme={null} $ helm repo add bitnami https://charts.bitnami.com/bitnami $ helm install my-release bitnami/wordpress
```

This concludes our overview of Helm Charts. In the next lesson, we will delve deeper into chart templating techniques and explore advanced methods to customize your Kubernetes deployments.



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Working with Helm basics

This article covers basic Helm operations using the command-line interface for managing Kubernetes applications and chart repositories.

This article explains basic Helm operations using the command-line interface. When you run the Helm command without any subcommands, it provides a list of common actions and available commands.

For example, executing:

```
```plaintext theme={null} $ helm --help
```

displays:

```
```plaintext theme={null}
```

The Kubernetes package manager

Common actions for Helm:

- helm search: search for charts
- helm pull: download a chart to your local directory to view
- helm install: upload the chart to Kubernetes
- helm list: list releases of charts

Usage:

```
helm [command]
```

Available Commands:

completion	generate autocompletion scripts for the specified shell
create	create a new chart with the given name
dependency	manage a chart's dependencies
env	helm client environment information
get	download extended information of a named release
help	Help about any command
history	fetch release history

This output serves as a quick reference to the available Helm commands. For instance, if you need to revert a failed upgrade, you can quickly find out by searching within the help details that the correct command is `helm rollback`, not “restore.”

## Managing Chart Repositories

Exploring repository-related commands is straightforward. Running:

```
``plaintext theme={null} $ helm repo --help
```

produces:

```
``plaintext theme={null}
```

This command consists of multiple subcommands to interact with chart repositories.

It can be used to add, remove, list, and index chart repositories.

Usage:

```
helm repo [command]
```

Available Commands:

```
add add a chart repository
index generate an index file given a directory containing packaged charts
list list chart repositories
remove remove one or more chart repositories
update update information of available charts locally from chart repositories
```

This clearly outlines the commands for managing chart repositories—whether you need to add, list, remove, or update them.

## Deploying an Application on Kubernetes

Imagine you want to launch a WordPress website on Kubernetes using a chart from an online repository, such as [Artifact Hub](#). Charts with an official or verified publisher badge ensure authenticity and reliability. Follow these steps:

1. **Note:** First, add the Bitnami repository:

```
bash theme={null} $ helm repo add bitnami https://charts.bitnami.com/bitnami
```

1. **Note:** Then, install the WordPress chart:

```
bash theme={null} $ helm install my-release bitnami/wordpress
```

After running these commands, you should see an output similar to:

```
``bash theme={null} $ helm repo add bitnami https://charts.bitnami.com/bitnami "bitnami" has been added to your repositories
```

```
$ helm install my-release bitnami/wordpress NAME: my-release LAST DEPLOYED: Wed Nov 10 18:03:50 2021
NAMESPACE: default STATUS: deployed REVISION: 1 TEST SUITE: None NOTES: CHART NAME: wordpress
CHART VERSION: 12.1.27 APP VERSION: 5.8.1
```

**\*\* Please be patient while the chart is being deployed \*\*** Your WordPress site can be accessed through the following DNS name from within your cluster: my-release-wordpress.default.svc.cluster.local (port 80)

```
This output confirms that your WordPress application is successfully deployed and provides details on how to access it.

Searching for Charts

Helm offers robust search functionality to help you find specific charts. To search for a chart on Artifacts Hub, run the following command:

```bash theme={null}
$ helm search hub wordpress
```

This command returns a list of available WordPress charts with details such as chart version, app version, and a short description. If you prefer to search within your local repositories, simply run:

```
```plaintext theme={null} $ helm search wordpress
```

```
This command provides similar information that helps you identify the correct chart version corresponding to your requirements.

Managing Helm Releases

Once a chart is deployed, it is managed as a release. To view all existing releases, run:

```bash theme={null}
$ helm list
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART
my-release	default	1	2021-11-10 18:03:50.414174217 +0000 UTC	deployed	wordpress-12.1.2

If you later decide to remove the deployed release (for instance, the WordPress website), you can uninstall it with:

```
```bash theme={null} $ helm uninstall my-release
```

```
This command cleans up all Kubernetes objects that were created during deployment.

Managing Local Helm Repository Data

To view your configured chart repositories, execute:

```plaintext theme={null}
$ helm repo list
```

NAME	URL
bitnami	https://charts.bitnami.com/bitnami

Over time, locally cached repository data can become outdated. To refresh this information, run:

```
plaintext theme={null} $ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
Update Complete. 🎉 Happy Helming! 🎉
```

This command ensures you have the latest chart data locally.

Wrapping Up: This article provided an overview of basic Helm CLI operations—from accessing help information to deploying and managing applications on Kubernetes. Experiment with these commands in your environment to become more confident with Helm.

Happy Helming!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Customizing chart parameters

Learn to customize chart parameters during a Helm chart installation, including overriding default values for WordPress using command-line options or custom values files.

In this guide, you'll learn how to customize chart parameters during a Helm chart installation. When you deploy WordPress using the Bitnami chart, it uses the default values defined in the chart's `values.yaml` file. For example, the default blog name is set as "User's Blog!" in the values file. This article explains how this value is configured and outlines the various methods available for overriding it.

Understanding the Default Configuration

The WordPress application is deployed using a Kubernetes Deployment resource. Its configuration is partly derived from the values set in `values.yaml`. Below is a snippet from the `values.yaml` file indicating the default settings:

```
``yaml theme={null}
```

values.yaml snippet

```
image: registry: docker.io repository: bitnami/wordpress tag: 5.8.2-debian-10-r0
```

@param wordpressUsername WordPress username

```
wordpressUsername: user
```

@param wordpressPassword WordPress user password

Defaults to a random 10-character alphanumeric string if not set

wordpressPassword: ""

@param existingSecret

existingSecret: ""

@param wordpressEmail WordPress user email

wordpressEmail: user@example.com

@param wordpressBlogName Blog name

wordpressBlogName: User's Blog!

The corresponding Deployment template uses these values to configure the environment variables. For example,

```
```yaml theme={null}
Deployment template snippet
apiVersion: {{ include "apiVersion" }}
kind: Deployment
metadata:
 name: {{ include "common.names.fullname" }}
 namespace: {{ .Release.Namespace | quote }}
 labels: {{- include "common.labels.standard" | nindent 4 }}
spec:
 selector:
 matchLabels: {{- include "common.labels.matchLabels" }}
 replicas: {{ .Values.replicaCount }}
 template:
 spec:
 containers:
 - name: wordpress
 image: {{ template "wordpress.images" . }}
 env:
 - name: WORDPRESS_DATABASE_NAME
 value: {{ include "wordpress.databaseName" | quote }}
 - name: WORDPRESS_DATABASE_USER
 value: {{ include "wordpress.databaseUser" | quote }}
 - name: WORDPRESS_USERNAME
 value: {{ .Values.wordpressUsername | quote }}
 - name: WORDPRESS_PASSWORD
 valueFrom:
 secretKeyRef:
 name: {{ include "wordpress.secretName" }}
 key: wordpress-password
 - name: WORDPRESS_BLOG_NAME
 value: {{ .Values.wordpressBlogName | quote }}
```

The `WORDPRESS_BLOG_NAME` environment variable is directly set using the value from `values.yaml`, meaning that the application will deploy with "User's Blog!" as the default blog name unless it's overridden.

## Overriding Default Values with Command-Line Parameters

When you install WordPress with Helm, the chart deploys using the default values from the `values.yaml` file. To override these defaults on the fly, you can use the `--set` option with the `helm install` command. For example, to change the blog name from "User's Blog!" to "Helm Tut", use the following command:

```
bash theme={null} $ helm install --set wordpressBlogName="Helm Tut" my-release bitnami/wordpress
```

You can override multiple parameters simultaneously. For instance, you may adjust both the WordPress blog name and the user email. Command-line parameters provided using `--set` will take precedence over the defaults defined in the `values.yaml` file.

## Using a Custom Values File

For numerous parameter overrides, maintaining a custom values file is more efficient than using multiple `--set` options. Follow these steps:

1. Create a file named `custom-values.yaml` with your custom configurations:

```
yaml theme={null} # custom-values.yaml snippet wordpressBlogName: Helm Tut wordpressEmail: john@example.com
```

1. Deploy the chart using the custom values file by running:

```
bash theme={null} $ helm install my-release bitnami/wordpress --values custom-values.yaml
```

This instructs Helm to apply the configuration from `custom-values.yaml`, thereby overriding the corresponding default values.

## Modifying the Built-in values.yaml File

If you wish to modify the built-in `values.yaml` within the chart itself, you can do so by following these steps:

1. Use `helm pull` to download the chart in an archived (compressed) form:

```
bash theme={null} $ helm pull bitnami/wordpress
```

1. To automatically uncompress the chart into a directory, use the `--untar` option:

```
bash theme={null} $ helm pull --untar bitnami/wordpress
```

1. List the directory to view the chart files, including `values.yaml`:

```
``bash theme={null} $ ls wordpress
```

```
$ ls wordpress ci templates Chart.lock Chart.yaml README.md values.schema.json values.yaml ``
```

1. Open and edit the `values.yaml` file in your preferred text editor to set your custom configurations.
2. Install the chart locally by referencing the modified chart path:

```
bash theme={null} $ helm install my-release ./wordpress
```

In this command, `./` indicates the current directory, and Helm installs the chart using your modified files.

When modifying the built-in `values.yaml` file, ensure you maintain the correct file structure to avoid deployment issues.

---

This concludes our discussion on customizing chart parameters. You have learned how to override default values in a Bitnami WordPress Helm chart using command-line options with `--set`, by employing a custom values file, or by directly modifying the chart's source after pulling it locally. For more details on Helm and managing Kubernetes resources, explore the [official Helm documentation](#) and [Kubernetes Basics](#).



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Lifecycle management with Helm

This article explores managing Kubernetes application lifecycles using Helm, covering releases, upgrades, and rollbacks with practical examples.

In this article, we explore how to effectively manage the lifecycle of Kubernetes applications using Helm. Learn how Helm handles releases, upgrades, and rollbacks through real-world examples that simplify complexity and enhance application management.

## Creating and Managing Releases

When you install a Helm chart, a release is created. Each release is like an application package—a collection of related Kubernetes objects. Since Helm tracks all objects associated with a release, it allows you to upgrade, downgrade, or uninstall a release without affecting other deployments. For instance, even if you deploy the same chart twice, each release remains independent:

```
```bash theme={null} $ helm install my-site bitnami/wordpress $ helm install my-SECOND-site bitnami/wordpress
```

Installing an Older Version

To see Helm in action, let's create a new release by installing an older version of the NGINX chart. Use the

```
```bash theme={null}
$ helm install nginx-release bitnami/nginx --version 7.1.0
```

This command deploys an NGINX release named "nginx-release" using an earlier version of NGINX. After installation, verify the Pod status and details of the image:

```
```bash theme={null} $ kubectl get pods NAME READY STATUS RESTARTS AGE nginx-release-687cdd5c75-ztn2n 0/1 ContainerCreating 0 13s
```

Once the Pod is running, get detailed information about the image:

```
```bash theme={null}
$ kubectl describe pod nginx-release-687cdd5c75-ztn2n
Containers:
 nginx:
 Container ID: docker://81bb5ad6b5...
 Image: docker.io/bitnami/nginx:1.19.2-debian-10-r28
 Image ID: docker-pullable://bitnami/nginx@sha256:2fcaf026b8acb7a...
 Port: 8080/TCP
 Host Port: 0/TCP
 State: Running
```

In this case, the installed NGINX version is 1.19.2, which might become outdated over time.

If you discover security vulnerabilities or need feature updates, Helm makes it easy to upgrade your application along with all associated Kubernetes configurations.

## Upgrading Releases

Helm's upgrade functionality allows seamless transition to new versions. When you upgrade a release, Helm replaces the old Pod with a new one that includes updated settings and images. Here's an example of upgrading the existing NGINX release:

```
```bash theme={null} $ helm upgrade nginx-release bitnami/nginx
Release "nginx-release" has been upgraded.
Happy Helming! NAME: nginx-release LAST DEPLOYED: Mon Nov 15 19:25:55 2021 NAMESPACE: default
STATUS: deployed REVISION: 2 TEST SUITE: None NOTES: CHART NAME: nginx CHART VERSION: 9.5.13
APP VERSION: 1.21.4
```

After upgrading, verify the updates with the following commands:

```
```bash theme={null}
$ helm list
```

NAME	NAMESPACE	REVISION	STATUS	CHART	APP VERSION
nginx-release	default	2	deployed	nginx-9.5.13	1.21.4

Check the revision history to get more insights into the changes:

```
```bash theme={null} $ helm history nginx-release
REVISION UPDATED STATUS CHART APP VERSION
DESCRIPTION 1 Mon Nov 15 19:20:51 2021 superseded nginx-7.1.0 1.19.2 Install complete
2 Mon Nov 15 19:25:55 2021 deployed nginx-9.5.13 1.21.4 Upgrade complete
```

Rollbacks

If an upgrade leads to unexpected behavior, Helm supports rollbacks. Rolling back reverts the release to

```
```bash theme={null}
$ helm rollback nginx-release 1
Rollback was a success! Happy Helming!
```

After a rollback, the configuration reverts to the settings in revision 1. However, note that Helm records this as a new revision, providing a complete history for audit and troubleshooting.

Remember that while rollbacks restore Kubernetes manifest configurations, they do not include the actual data stored in persistent volumes or external databases. Always ensure you have a separate backup solution for persistent data.

## Upgrading Complex Charts

When working with more complex applications, such as WordPress, additional parameters might be required during an upgrade. Missing these parameters can result in errors like the one below:

```
```bash theme={null} $ helm upgrade wordpress-release bitnami/wordpress Error: UPGRADE FAILED: template:
wordpress/templates/NOTES.txt:83:4: executing "wordpress/templates/NOTES.txt" at : error calling include:
template:                wordpress/charts/common/templates/_errors.tpl:21:48:                executing
"common.errors.upgrade.passwords.empty" at : error calling fail: PASSWORDS ERROR: You must provide your
current passwords when upgrading the release. Note that even after reinstallation, old credentials may be
needed as they may be kept in persistent volume claims. Further information can be obtained at https://
docs.bitnami.com/general/how-to/troubleshoot-helm-chart-issues/#credential-errors-while-upgrading-chart-
releases
```

'wordpressPassword' must not be empty, please add '--set wordpressPassword=\$WORDPRESS_PASSWORD' to the command. To get the current value: export WORDPRESS_PASSWORD=\$(kubectl get secret --namespace "default" wordpress-release -o jsonpath="{.data.wordpress-password}" | base64 --decode)

'mariadb.auth.rootPassword' must not be empty, please add '--set mariadb.auth.rootPassword=\$MARIADB_ROOT_PASSWORD' to the command. To get the current value: export MARIADB_ROOT_PASSWORD=\$(kubectl get secret --namespace "default" wordpress-release-mariadb -o jsonpath="{.data.mariadb-root-password}" | base64 --decode)

'mariadb.auth.password' must not be empty, please add '--set mariadb.auth.password=\$MARIADB_PASSWORD' to the command. To get the current value: export MARIADB_PASSWORD=\$(kubectl get secret --namespace "default" wordpress-release-mariadb -o jsonpath="{.data.mariadb-password}" | base64 --decode) ```

This error indicates that administrative credentials must be provided for certain components during the upgrade. Always supply the necessary parameters to ensure that all Kubernetes objects and application components are appropriately updated.

Summary

Helm simplifies lifecycle management by:

Action	Description	Command Example
Install	Create a new release from a Helm chart.	helm install my-release bitnami/nginx --version 7.1.0
Upgrade	Update an existing release to a new version with all associated configurations.	helm upgrade my-release bitnami/nginx
Rollback	Revert a release to a previous configuration state in case of issues.	helm rollback my-release 1

By following the practices outlined in this article, you can streamline application management across your Kubernetes clusters. Practice these Helm commands with hands-on exercises to deepen your understanding and improve your deployment workflows.

For more information, consider exploring the [Helm Documentation](#).